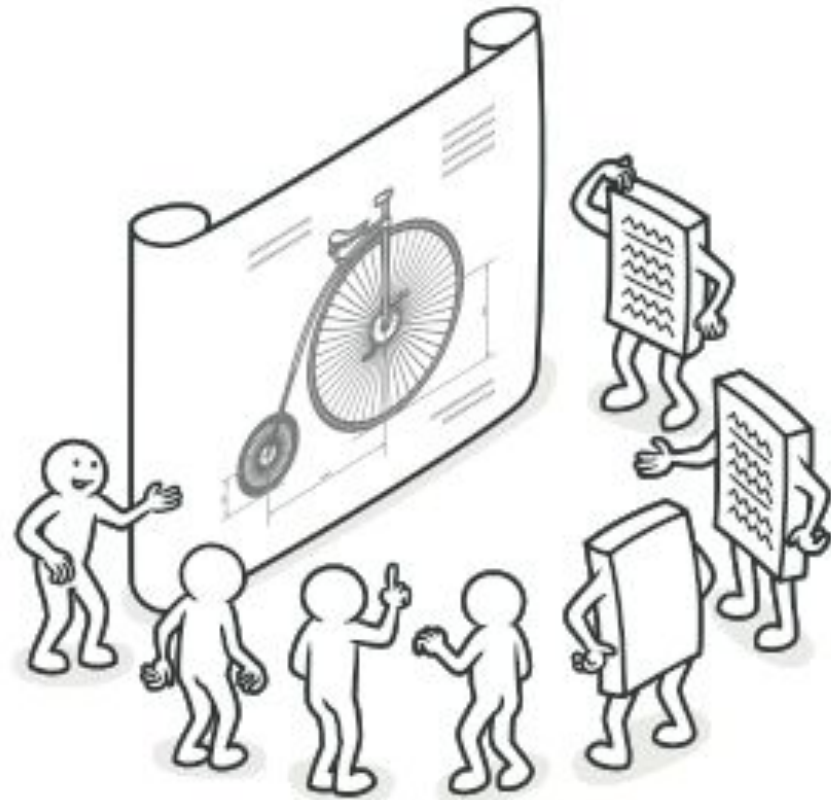


LICENCIATURA EN
CIENCIA DE DATOS

PROGRAMACIÓN AVANZADA (189)

Lic. Felipe Morales, APU Gianluca Piriz
Clase N. 10. Unidad II. Patrones de diseño



TEMARIO

- T0: **Revisión** de Clase anterior
 - Revisión de dudas
 - Análisis de resoluciones de estudiantes
 - Conclusiones
- T1: Patrones de diseño
- T2: Actividad práctica
- **Temas relacionados**
- **Links**

T0: Revisión de **Clase Anterior**:

- Revisión de dudas
- Análisis de resoluciones de estudiantes
- Presentación de Trabajo Grupal (10 minutos máximo y 5 de feedback)
- Conclusiones

T1: Patrones de diseño

Introducción:

Los **patrones de diseño (design patterns)** son soluciones habituales a problemas comunes en el diseño de software. Cada **patrón** es como un plano que se puede personalizar para resolver un problema de diseño particular de tu código.

Ventajas:

Los patrones son un juego de herramientas que brindan soluciones a problemas habituales en el **diseño de software**. Definen un lenguaje común que ayuda a tu equipo a comunicarse con más eficiencia.

T1: Patrones de diseño

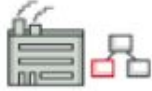
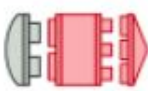
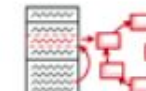
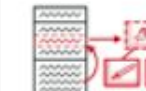
¿Por qué debería aprender sobre patrones?:

La realidad es que podrías trabajar durante años como programador sin conocer un solo patrón. Mucha gente lo hace. Incluso en ese caso, podrías estar implementando patrones sin saberlo. Así que, ¿por qué dedicar tiempo a aprenderlos?

Los patrones de diseño son un juego de herramientas de soluciones comprobadas a problemas habituales en el diseño de software. Incluso aunque nunca te encuentres con estos problemas, conocer los patrones sigue siendo de utilidad, porque te enseña a resolver todo tipo de problemas utilizando principios del diseño orientado a objetos.

T1: Patrones de diseño

22 ejemplos:

							
Factory Method	Abstract Factory	Adapter	Bridge	Chain of Responsibility	Command	Iterator	Mediator
							
Builder	Prototype	Composite	Decorator	Memento	Observer	State	Strategy
							
Singleton		Facade	Flyweight	Template Method	Visitor		

T1: Patrones de diseño

Clasificación:

Los patrones de diseño varían en su complejidad, nivel de detalle y escala de aplicabilidad al sistema completo que se diseña. Me gusta la analogía de la construcción de carreteras: puedes hacer más segura una intersección instalando semáforos o construyendo un intercambiador completo de varios niveles con pasajes subterráneos para peatones.

Los patrones más básicos y de más bajo nivel suelen llamarse idioms. Normalmente se aplican a un único lenguaje de programación.

Los patrones más universales y de más alto nivel son los patrones de arquitectura. Los desarrolladores pueden implementar estos patrones prácticamente en cualquier lenguaje. Al contrario que otros patrones, pueden utilizarse para diseñar la arquitectura de una aplicación completa.

T1: Patrones de diseño

Clasificación:

Además, todos los patrones pueden clasificarse por su propósito. Este libro cubre tres grupos generales de patrones:

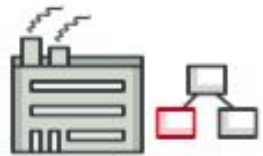
Los **patrones creacionales** proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización de código existente.

Los **patrones estructurales** explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que se mantiene la flexibilidad y eficiencia de la estructura.

Los **patrones de comportamiento** se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos.

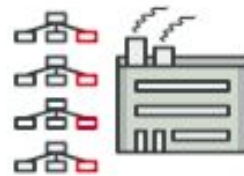
T1: Patrones de diseño

Patrones creacionales



Factory Method

Proporciona una interfaz para la creación de objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.



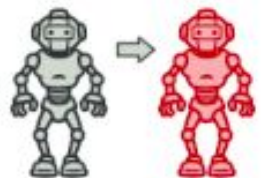
Abstract Factory

Permite producir familias de objetos relacionados sin especificar sus clases concretas.



Builder

Permite construir objetos complejos paso a paso. Este patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.



Prototype

Permite copiar objetos existentes sin que el código dependa de sus clases.

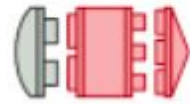


Singleton

Permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.

T1: Patrones de diseño

Patrones estructurales



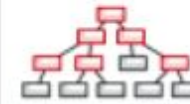
Adapter

Permite la colaboración entre objetos con interfaces incompatibles.



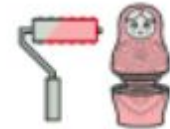
Bridge

Permite dividir una clase grande o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una de la otra.



Composite

Permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.



Decorator

Permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.



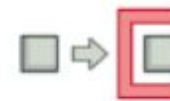
Facade

Proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases.



Flyweight

Permite mantener más objetos dentro de la cantidad disponible de memoria RAM compartiendo las partes comunes del estado entre varios objetos en lugar de mantener toda la información en cada objeto.



Proxy

Permite proporcionar un sustituto o marcador de posición para otro objeto. Un proxy controla el acceso al objeto original, permitiéndote hacer algo antes o después de que la solicitud llegue al objeto original.

T1: Patrones de diseño

Patrones de comportamiento (parte i)



Chain of Responsibility

Permite pasar solicitudes a lo largo de una cadena de manejadores. Al recibir una solicitud, cada manejador decide si la procesa o si la pasa al siguiente manejador de la cadena.



Command

Convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.



Iterator

Permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).



Mediator

Permite reducir las dependencias caóticas entre objetos. El patrón restringe las comunicaciones directas entre los objetos, forzándolos a colaborar únicamente a través de un objeto mediador.



Memento

Permite guardar y restaurar el estado previo de un objeto sin revelar los detalles de su implementación.

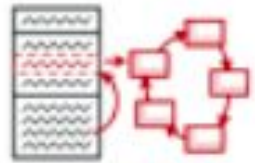


Observer

Permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

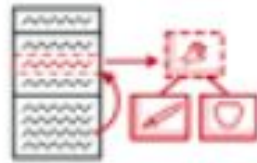
T1: Patrones de diseño

Patrones de comportamiento (parte ii)



State

Permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiara su clase.



Strategy

Permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.



Template Method

Define el esqueleto de un algoritmo en la superclase pero permite que las subclasses sobrescriban pasos del algoritmo sin cambiar su estructura.



Visitor

Permite separar algoritmos de los objetos sobre los que operan.

T1: Patrones de diseño

Actividad práctica

Ejercicio 1:

Investigar y documentar críticas a los patrones de diseño. Mencione ejemplos concretos.

Ejercicio 2:

Seleccione 3 patrones de diseño e implementarlos en Python. Arme ejemplos concretos de uso. Lo ideal es elegir un patrón de cada clasificación.

Ejercicio 3:

Piense en 3 problemas habituales de su vida diaria en los cuales podría aplicar patrones de diseño.

T1: Patrones de diseño

Actividad práctica

Ejercicio 4:

Los patrones de diseño suelen poseer distintos nombres o denominaciones. Busque y arme una tabla con los posibles distintos nombres usados.

Ejercicio 5:

¿Qué son los antipatrones de diseño? Ejemplifique algunos casos.

Ejercicio 6 (opcional):

Investigue el uso de otras buenas prácticas, como por ejemplo SOLID.

Links de Material Útil sobre patrones de diseño:

- <https://refactoring.guru/es/design-patterns/what-is-pattern>
- <https://refactoring.guru/es/design-patterns/python>

Principios SOLID:

- <https://profile.es/blog/principios-solid-desarrollo-software-calidad/>

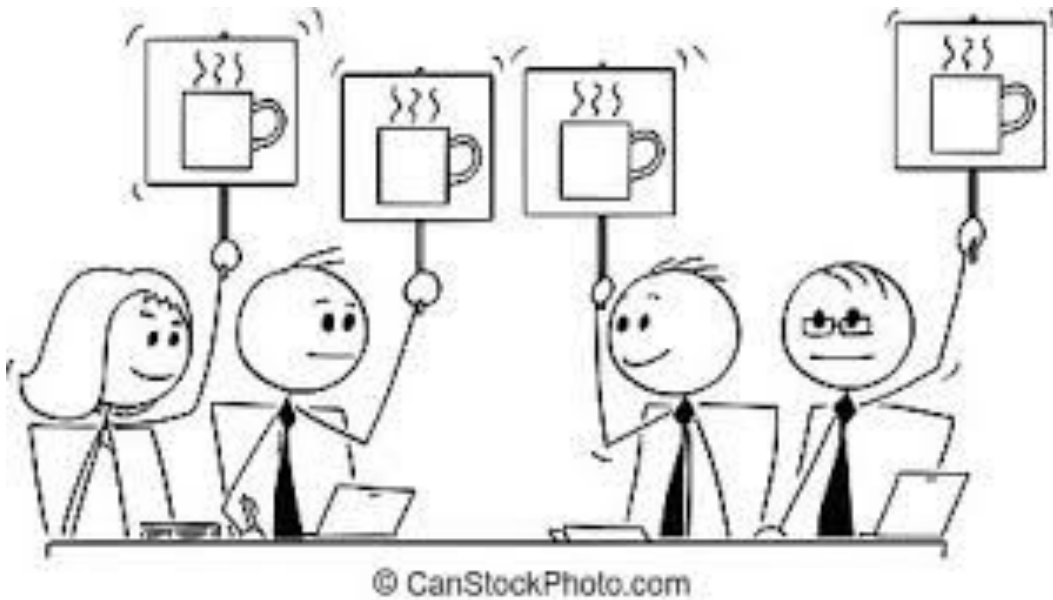
Colab con ejemplos:

- <https://drive.google.com/file/d/1nvGbbA3wgDulk-lvacIRLeZl0JqxCCBz/view?usp=sharing>

A TRABAJAR



BREAK



¿CHALLENGE?



¿SEGUIMOS TRABAJANDO?

Links interesantes:

Aprenda a pensar como un programador con Python

<https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>

Documentación oficial:

<https://www.python.org/doc/>

Curso de Python desde 0 (pildorasinformaticas)

<https://www.youtube.com/playlist?list=PLU8oAIHdN5BlvPxziopYZRd55pdqFwkeS>

Otros links:

<https://ellibrodepython.com/polimorfismo-en-programacion>

<https://parzibyte.me/blog/2019/06/30/clases-constructores-python-poo/>

<https://elvex.ugr.es/idbis/db/docs/design/2-requirements.pdf>

¡Muchas gracias por tu atención!